

RAG System — Technical Specs

Roko APG Software • March 23, 2026

Tech Stack

Layer	Technology
Frontend	React 19, Next.js 16 (App Router), TypeScript, Tailwind CSS 4, shadcn/ui
Backend	Next.js API Routes (Node.js)
AI / LLM	Google Gemini 2.5 Flash / Pro
Embeddings	gemini-embedding-001 (3072-dimension vectors)
Database	Supabase (PostgreSQL) with pgvector extension
File Storage	Supabase Storage
PDF OCR	Docling (open-source parser) + Gemini Vision (Deep Extract)
Hosting	Railway (Docker containers)

Supported File Types

Category	Formats
Documents	PDF, DOCX, PPTX, XLSX
Images	JPG, JPEG, PNG, GIF, WEBP
Video	MP4, WEBM, MOV, AVI
Audio	MP3, WAV, OGG, FLAC, M4A, AAC
Text	TXT, MD, CSV, JSON, HTML

Processing Pipeline

When a file is uploaded, it goes through these stages:

Stage 1 — Text Extraction

- Docling (default):** Open-source parser, works well for digital PDFs with selectable text. No AI cost.
- Deep Extract (AI Vision):** Sends each PDF page as an image to Gemini for OCR. Best for scanned docs, complex layouts, tables. Processes in batches of 15 pages.

- **Non-PDF files:** Each file type has its own processor (image description via Gemini Vision, audio/video transcription via Gemini, etc.)

Stage 2 — Chunking

The extracted text is split into searchable chunks. Five strategies available:

Strategy	How It Works	AI Cost
Standard	Splits at headings, paragraphs, sentences	None
Contextual	Standard + prepends document context to each chunk	None
Semantic	Detects topic boundaries via embedding similarity	Embedding calls
AI Smart	LLM decides optimal split points	LLM calls
Page-Based	Splits by page boundaries (Deep Extract only)	None

Chunk sizes: Small (256 tokens), Medium (512), Large (1024), or Custom (100-1500).

Stage 3 — Enrichment (per chunk)

- **Embedding generation:** Every chunk gets a 768-dim vector via gemini-embedding-001
- **AI-Generated Context (optional):** One-sentence summary per chunk to improve search accuracy
- **Custom Metadata Extraction (optional):** Structured fields extracted per chunk (e.g., dates, names, categories)
- **Entity & Relation Extraction (optional):** Extracts people, organizations, concepts, and their relationships for graph-based search

Stage 4 — Storage

- Chunks stored in `chunks` table with `content_vector` (VECTOR(768))
- Entities stored with their own embeddings
- Relations stored linking entities together
- All scoped by project for multi-tenancy

Query / Search Modes

When a user asks a question, the system retrieves relevant chunks using one of these modes:

Mode	Method	Requires Entities
Naive	Pure vector similarity search	No
Local	Entity-centric search (finds chunks via matching entities)	Yes
Global	Relationship-based search (finds chunks via entity relationships)	Yes
Hybrid	Local + Global combined	Yes
Mix	All methods combined (naive + local + global)	Yes
Human in the Loop	Two-step: retrieve chunks, user approves, then generate answer	No

Agentic Mode (optional layer on top of any mode)

- Breaks question into sub-questions
- Runs multiple search rounds using the selected query mode
- Evaluates results after each round, stops early if enough info found
- Configurable: 2, 3, 5, or 7 max rounds

Processing Mode Presets

Mode	Extraction	Chunking	AI Context	Entities	Cost Estimate
Budget	Docling	Semantic	Yes	No	~\$2-4 / 1,600 pages
Standard	Deep Extract	Page-Based	Yes	No	~\$12-18 / 1,600 pages
Full	Deep Extract	Page-Based	Yes	Yes	~\$25-35 / 1,600 pages
Custom	Mix and match any settings				Varies

Multi-Tenancy & Authentication

- **Organizations:** Each org gets its own admin panel, API keys, and projects
- **Projects:** Documents are scoped to projects within an organization
- **User auth:** Cookie-based sessions (login/register)
- **Admin auth:** Separate session-based authentication
- **API auth:** Bearer token (**Authorization: Bearer mrag_sk_xxx**)
- **API key encryption:** AES-256 with configurable encryption key

V1 API Endpoints

Method	Endpoint	Description
POST	<code>/api/v1/query</code>	Query documents with RAG
POST	<code>/api/v1/documents</code>	Upload a document
GET	<code>/api/v1/documents</code>	List documents
POST	<code>/api/v1/chat</code>	Create a chat session
POST	<code>/api/v1/chat/{id}/messages</code>	Send message in a chat

All V1 endpoints require API key authentication.

Cost Tracking

- Tracks real token usage from Gemini's `usageMetadata` (input tokens, output tokens, cost per call)
- Per-project monthly spending displayed in settings
- Configurable monthly spending cap — blocks new processing when reached, queries always work
- Cost breakdown visible per operation

Database Schema (Core Tables)

Table	Purpose
<code>organizations</code>	Tenant organizations
<code>projects</code>	Projects within an organization
<code>documents</code>	File metadata, status, processing flags
<code>chunks</code>	Text segments with <code>content_vector</code> (VECTOR(768))
<code>entities</code>	Extracted entities with embeddings
<code>relations</code>	Entity-to-entity relationships
<code>chat_sessions</code>	Conversation sessions
<code>chat_messages</code>	Individual messages with sources
<code>api_keys</code>	Encrypted API keys per organization
<code>usage_logs</code>	Token usage and cost tracking

PostgreSQL search functions: `search_chunks()`, `search_entities()`, `search_relations()` — all use pgvector for similarity search.

Infrastructure

- **Next.js app:** Main application (frontend + API)
 - **Docling service:** Separate Docker container running the Python-based PDF parser
 - **Supabase:** Managed PostgreSQL + Storage (hosted)
 - **Railway:** Container hosting with auto-deploy from GitHub
-

Key Features Summary

- Multi-modal: Process PDFs, docs, images, video, audio
 - 5 chunking strategies with configurable chunk sizes
 - 6 query modes including graph-based search and human-in-the-loop
 - Agentic RAG with multi-round iterative search
 - Conversation memory with query rewriting for follow-ups
 - Per-project settings, processing modes, and spending caps
 - White-label admin panel with API key management
 - V1 REST API for external integrations
 - Real-time cost tracking with monthly caps
 - Chunk health monitoring with targeted retry for failed enrichments
-

Generated with PDF Report Generator • 2026